Glossary

Book: Glossary, pp. 553–558

The book ends with a short glossary. Its author notes that the entries are deliberately brief: they capture the core idea of each term, not every subtlety, and point back to the main text for the details. The table below covers **every** glossary entry, rewritten in plain English, with links to the chapter notes where the term is explained properly. (The book's Index is not reproduced here.)

Chapter links: 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10 · 11 · 12

> "Part II intro" is summarised at the start of the Chapter 5 notes, and "Part III intro" at the start of the Chapter 10 notes.

Term	Plain-English meaning	Where it's explained (chapter)
asynchronous	Carrying on without waiting for something to finish (for example, a message being sent to another node over the network), and making no assumption about how long that thing will take.	Ch 5 (synchronous vs asynchronous replication), Ch 8 (synchronous vs asynchronous networks; system models)
atomic	Has two meanings. (1) For concurrent operations: the operation seems to happen at one instant, so no other concurrent process can ever catch it half-done (compare *isolation*). (2) For transactions: a group of writes that is either committed as a whole or rolled back as a whole, even when faults happen.	Ch 7 (Atomicity), Ch 9 (atomic commit and 2PC)
backpressure	Making a data producer slow down because the receiver can't keep up. Another name for it is *flow control*.	Ch 11 (Messaging Systems)
batch process	A job that reads a fixed, usually large, input dataset and writes some other dataset as its output, leaving the input unchanged.	Ch 10
bounded	Having a known maximum limit or size. Used, for instance, about network delays and about datasets.	Ch 8 (timeouts and unbounded delays), Ch 11 (introduction)
Byzantine fault	A node misbehaving in an arbitrary way — for example, sending other nodes messages that contradict each other or are deliberately malicious.	Ch 8 (Byzantine Faults)
cache	A component that keeps recently used data so that later reads of the same data are quicker. It normally holds only part of the data, so anything missing must be fetched from a slower underlying store that has the full copy.	General term (the book cites no single section); see e.g. Ch 1, Ch 12
CAP theorem	A theoretical result that is widely misunderstood and, in the book's view, not useful in practice.	Ch 9 (The CAP theorem)
causality	The dependency that exists when one event "happens before" another: a later event replies to, builds on, or can only be understood in light of an earlier one.	Ch 5 (happens-before and concurrency), Ch 9 (Ordering and Causality)
consensus	The core distributed-computing problem of getting several nodes to agree on something, such as which node leads a database cluster. It is much harder than it first appears.	Ch 9 (Fault-Tolerant Consensus)
data warehouse	A database that combines data from several OLTP systems and prepares it for analytics.	Ch 3 (Data Warehousing)
declarative	Stating which properties the result should have, rather than the exact steps for producing it. For queries, a query optimizer receives the declarative query and chooses the best way to run it.	Ch 2 (Query Languages for Data)
denormalize	Deliberately adding some redundancy or duplication to a normalized dataset — usually as a cache or index — so reads are faster. A denormalized value is effectively a precomputed query result, much like a materialized view.	Ch 7 (single- and multi-object operations), Ch 11 (deriving several views from one event log)

Term	Plain-English meaning	Where it's explained (chapter)
derived data	A dataset produced from other data by a repeatable process that you could rerun if needed. It usually exists to make some specific kind of read fast. Indexes, caches and materialized views are examples.	Part III intro — Ch 10 overview; also Ch 12
deterministic	Describes a function that, given the same input, always returns the same output. It therefore can't rely on random numbers, the current time, network calls, or anything else unpredictable.	General term (no section cited); used in Ch 10, Ch 11, Ch 12
distributed	Running on several nodes that talk over a network. The hallmark is partial failure: one part may be broken while others work, and the software often can't tell exactly what has broken.	Ch 8 (Faults and Partial Failures)
durable	Stored in a way that you trust will survive various faults without being lost.	Ch 7 (Durability)
ETL	Extract–Transform–Load: pulling data out of a source database, reshaping it to suit analytic queries, and loading it into a data warehouse or batch processing system.	Ch 3 (Data Warehousing)
failover	In a single-leader system, handing the leader role over from one node to another.	Ch 5 (Handling Node Outages)
fault-tolerant	Able to recover by itself when something breaks, such as a machine crashing or a network link going down.	Ch 1 (Reliability)
flow control	Same thing as *backpressure*.	Ch 11
follower	A replica that never takes writes directly from clients; it only applies the data changes it receives from a leader. Also called a secondary, slave, read replica or hot standby.	Ch 5 (Leaders and Followers)
full-text search	Searching text for arbitrary keywords, often with extras like matching words that are spelled similarly, or synonyms. A full-text index is a type of secondary index that supports such searches.	Ch 3 (full-text search and fuzzy indexes)
graph	A data structure made of vertices (things you can refer to; also called nodes or entities) and edges (links from one vertex to another; also called relationships or arcs).	Ch 2 (Graph-Like Data Models)
hash	A function that converts an input into a number that looks random. The same input always yields the same number. Different inputs almost always yield different numbers, but occasionally two inputs give the same output — a *collision*.	Ch 6 (Partitioning by Hash of Key)
idempotent	An operation that is safe to retry: running it several times has the same effect as running it once.	Ch 11 (Idempotence)
index	A data structure that lets you quickly find all records having a given value in a given field.	Ch 3 (Data Structures That Power Your Database)

Term	Plain-English meaning	Where it's explained (chapter)
isolation	For transactions: how much transactions running at the same time can interfere with one another. Serializable isolation is the strongest guarantee, but weaker levels are also used.	Ch 7 (Isolation)
join	Combining records that share something. Most often, one record refers to another (via a foreign key, a document reference, or a graph edge) and the query must fetch the record being referred to.	Ch 2 (many-to-one and many-to-many relationships), Ch 10 (reduce-side joins and grouping)
leader	When data or a service is replicated over several nodes, the one replica designated as allowed to make changes. It may be chosen by an election protocol or appointed by an administrator. Also called primary or master.	Ch 5 (Leaders and Followers)
linearizable	Acting as though the system held just one copy of the data, changed only by atomic operations.	Ch 9 (Linearizability)
locality	A performance technique: storing pieces of data together when they are often needed together.	Ch 2 (data locality for queries)
lock	A mechanism guaranteeing that only one thread, node or transaction can use something at a time; anyone else wanting it has to wait until it is released.	Ch 7 (Two-Phase Locking), Ch 8 (the leader and the lock)
log	A file you can only append to, used to store data. Examples: a write-ahead log makes a storage engine crash-safe; a log-structured storage engine uses logs as its main storage format; a replication log copies writes from leader to followers; an event log can represent a data stream.	Ch 3 (B-tree reliability; SSTables and LSM-Trees), Ch 5 (Leaders and Followers), Ch 11 (Partitioned Logs)
materialize	To compute something eagerly and store the result, instead of calculating it only when someone asks.	Ch 3 (data cubes and materialized views), Ch 10 (materialization of intermediate state)
node	One running instance of some software on a computer, which communicates with other nodes over a network to get a job done.	General term (no section cited); used throughout Part II (Ch 5–Ch 9)
normalized	Organised so that nothing is stored redundantly. When a fact changes in a normalized database, you update it in exactly one place instead of in many copies.	Ch 2 (many-to-one and many-to-many relationships)
OLAP	Online analytic processing: a workload dominated by aggregations (count, sum, average, etc.) across very many records.	Ch 3 (Transaction Processing or Analytics?)
OLTP	Online transaction processing: a workload of quick queries that each read or write a few records, typically looked up by key.	Ch 3 (Transaction Processing or Analytics?)
partitioning	Breaking a dataset or computation that is too big for one machine into smaller pieces spread across many machines. Also called sharding.	Ch 6
percentile	A way to describe how values are distributed by counting how many fall above or below a threshold. E.g., the 95th percentile response time over some period is the time *t* such that 95% of requests in	Ch 1 (Describing Performance)

Term	Plain-English meaning	Where it's explained (chapter)
	that period finished faster than *t* and 5% took longer.	
primary key	A value (usually a number or string) that uniquely identifies one record. In many applications the system generates it when the record is created (e.g., sequentially or at random), rather than a user choosing it. Contrast with *secondary index*.	General term (no section cited); see Ch 3
quorum	The minimum number of nodes that must vote for an operation before it counts as successful.	Ch 5 (quorums for reading and writing)
rebalance	Moving data or services between nodes so the load is spread evenly.	Ch 6 (Rebalancing Partitions)
replication	Keeping copies of the same data on several nodes (replicas) so it is still reachable when a node can't be reached.	Ch 5
schema	A description of how some data is structured, including its fields and their types. Data can be checked against a schema at different points in its life, and a schema may change over time.	Ch 2 (schema flexibility in the document model), Ch 4
secondary index	An extra data structure maintained next to the main data store that lets you efficiently find records matching some condition.	Ch 3 (Other Indexing Structures), Ch 6 (Partitioning and Secondary Indexes)
serializable	A guarantee that transactions running concurrently produce the same outcome as if they had run one after another in some order.	Ch 7 (Serializability)
shared-nothing	An architecture of independent nodes, each with its own CPU, memory and disks, linked by an ordinary network — as opposed to shared-memory or shared-disk designs.	Part II intro — Ch 5 overview
skew	Two meanings. (1) Uneven load across partitions: some get lots of data or requests while others get little; also called hot spots. (2) A timing anomaly that makes events appear in an unexpected, out-of-sequence order — see read skew, write skew and clock skew.	(1) Ch 6 (skewed workloads and hot spots), Ch 10 (handling skew); (2) Ch 7 (read skew in snapshot isolation; write skew and phantoms), Ch 8 (clock skew in timestamps for ordering events)
split brain	Two nodes both believing, at the same moment, that they are the leader — which can break the system's guarantees.	Ch 5 (Handling Node Outages), Ch 8 (the truth is defined by the majority)
stored procedure	Packaging a transaction's logic so it runs entirely on the database server, with no back-and-forth to the client during the transaction.	Ch 7 (Actual Serial Execution)
stream process	A computation that keeps running forever, reading an endless stream of events and producing some output from them.	Ch 11
synchronous	The opposite of *asynchronous*: waiting for the thing to complete.	Ch 5, Ch 8

Term	Plain-English meaning	Where it's explained (chapter)
system of record	The system that holds the main, authoritative version of some data — the "source of truth". Changes go here first, and other datasets can be derived from it.	Part III intro — Ch 10 overview; also Ch 12
timeout	One of the simplest ways to detect a fault: noticing that no response arrived within some time. You can't tell, though, whether the remote node failed or the network had a problem.	Ch 8 (Timeouts and Unbounded Delays)
total order	An ordering of items (e.g., timestamps) in which every pair can be compared, so you can always tell which of the two comes first. If some pairs can't be compared, it is called a *partial order*.	Ch 9 (the causal order is not a total order)
transaction	Bundling several reads and writes into one logical unit so that error handling and concurrency become simpler to deal with.	Ch 7
two-phase commit (2PC)	An algorithm that makes sure several database nodes either all commit a transaction or all abort it.	Ch 9 (Atomic Commit and Two-Phase Commit)
two-phase locking (2PL)	An algorithm for serializable isolation in which a transaction takes a lock on every piece of data it reads or writes and keeps those locks until it finishes.	Ch 7 (Two-Phase Locking)
unbounded	Having no known maximum limit or size; the opposite of *bounded*.	Ch 8, Ch 11